The BE-SNN-1 classifies a given input sample $\mathbf{x}$ based on the distance from extracted input features $\Phi_\theta(\mathbf{x})$ to the feature representation $\mathbf{E}_c$ of each class. Here, we adopt the average squared Euclidean distance $\bar{\mathbf{d}}_c(\mathbf{x}) = \frac{1}{M}\sum_{m=1}^{M} ||\Phi_\theta(\mathbf{x}) - \mathbf{e}_{c,m}||_2^2$ as the distance metric, where $||\cdot||_2$ denotes the L_2-norm. Each time BE-SNN-1 makes a prediction, the class feature extractor will generate a set of feature vectors $\mathbf{E}_c := \{\mathbf{e}_{c,m}\}_{m=1}^{M}$ by drawing M random samples $\{\epsilon_m\}_{m=1}^{M}$ from the standard Gaussian distribution $\mathcal{N}(\mathbf{0}, \mathbb{I}_{d_\epsilon})$, and classify the input $\mathbf{x}$ as the class with the minimum distance to the feature vector of the input, which is equivalent to the class with the maximum kernel value:

$$\underset{c}{\operatorname{argmax}}(\exp(-\bar{\mathbf{d}}_c(\mathbf{x}))) = \underset{c}{\operatorname{argmax}}\big\{ \exp(-\frac{1}{M}\sum_{m=1}^{M} ||\Phi_\theta(\mathbf{x}) - \mathbf{e}_{c,m}||_2^2)\big\} \ . \tag{7}$$

However, as discussed in Section 1, evaluating the uncertainty by modelling the distribution of data features may suffer from feature collapse. Therefore, we now discuss how to utilize the batch-ensemble mechanism to avoid feature collapse and introduce the proposed batch-ensemble stochastic neural networks (BE-SNNs).

Firstly, we build an ensemble of BE-SNN-1s by using batch-ensemble layers introduced in Section 2.2, thus each batch-ensemble layer has their own trainable vectors $\{r_{l,n_e}\}_{n_e=1}^{N_e}$ and $\{s_{l,n_e}\}_{n_e=1}^{N_e}$, where l is the layer index, and N_e is the ensemble size. Since the weights defined in Equation (2) are dependent on the trainable vectors $\{r_{l,n_e}\}_{n_e=1}^{N_e}$ and $\{s_{l,n_e}\}_{n_e=1}^{N_e}$, the output layer of BE-SNNs has N_e different outputs. As a result, the data feature extractors $\{\Phi_{\theta_{n_e}}(\cdot)\}_{n_e=1}^{N_e}$ in BE-SNNs produce N_e different feature vectors for a given input $\mathbf{x}$. And the class feature generators $\{\Psi_{\theta_{n_e}}(\cdot)\}_{n_e=1}^{N_e}$ produce $N_e \times M$ feature vectors for each class.

To make a prediction for a given input $\mathbf{x}$, the BE-SNNs classify $\mathbf{x}$ as the class with the maximum average kernel value:

$$\underset{c}{\operatorname{argmax}}\big\{ \frac{1}{N_e}\sum_{n_e=1}^{N_e} \exp(-\frac{1}{M}\sum_{m=1}^{M} ||\Phi_{\theta_{n_e}}(\mathbf{x}) - \mathbf{e}_{n_e,c,m}||_2^2)\big\} \ . \tag{8}$$

Loss functions of BE-SNNs. We first introduce the loss function for an individual ensemble member of BE-SNNs. The loss function of BE-SNN-1 consists of two parts, a classification loss and regularization terms. For the classification loss, we use the cross entropy loss between the exponentiated negative distance $\tilde{\mathbf{y}}_c = \exp(-\bar{\mathbf{d}}_c(\mathbf{x}))$ and the one-hot ground-truth class label $\mathbf{y}$:

$$L_{\text{cls}}(\tilde{\mathbf{y}}, \mathbf{y}) = -\sum_{c=1}^{C} \mathbf{y}_c \log(\tilde{\mathbf{y}}_c) + (1 - \mathbf{y}_c)\log(1 - \tilde{\mathbf{y}}_c) \ , \tag{9}$$

where $\mathbf{y}_c$ indicates the c-th element of the one-hot ground-truth label $\mathbf{y}$.

Entropy-based Regularization. To prevent class feature vectors $\mathbf{E}_c := \{\mathbf{e}_{c,m}\}_{m=1}^{M}$ from degenerating to single point estimates, i.e. feature vectors $\mathbf{e}_{c,m}$ converging to almost the same value for all $m \in \{1, \cdots, M\}$, we add a regularization term $R_c(\cdot)$ into the loss function. Particularly, we design the regularization term $R_c(\cdot)$ to encourage the entropy of the c-th class feature to be proportional to that of the data sample feature belonging to the c-th class. To achieve this, we first approximate the entropy $\hat{\mathbb{H}}(\{\Phi_\theta(\mathbf{x})|\mathbf{y}_c = 1\})$ of the data sample feature for all $c \in \{1, \cdots, C\}$ using the k-nearest neighbour entropy estimator proposed in (Lombardi and Pant, 2016). The class-dependent entropy $\hat{\mathbb{H}}(\{\Phi_\theta(\mathbf{x})|\mathbf{y}_c = 1\})$ is then used as a threshold to penalise class features whose entropy is smaller than this threshold. The described regularization term can be computed as follows:

$$R_c(\mathbf{E}_c) = R_c(\{\mathbf{e}_{c,m}\}_{m=1}^{M}) \tag{10}$$

$$= \max\left(0,\ \hat{\mathbb{H}}(\{\Phi_\theta(\mathbf{x})|\mathbf{y}_c = 1\}) - \hat{\mathbb{H}}(\{\mathbf{e}_{c,m}\}_{m=1}^{M})\right) \ . \tag{11}$$

In addition, the loss function of BE-SNN-1 also includes a gradient penalty term $L_{\text{gp}}(\tilde{\mathbf{y}})$ to encourage sensitivity of the classifier as in (van Amersfoort et al., 2020):

$$L_{\text{gp}}(\tilde{\mathbf{y}}) = \left[\left\|\nabla_{\mathbf{x}} \sum_{c} \tilde{\mathbf{y}}_c\right\|_2^2 - 1\right]^2 \ , \tag{12}$$

where $\nabla_{\mathbf{x}} \sum_c \tilde{\mathbf{y}}_c$ is the derivative of $\sum_c \tilde{\mathbf{y}}_c$ w.r.t to the input $\mathbf{x}$. The overall loss function for the n_e-th ensemble member of the BE-SNN model is as follows:

$$L_{n_e} = L_{\text{cls}}(\tilde{\mathbf{y}}, \mathbf{y}) + \lambda_1 \sum_{c=1}^{C} R_c(\mathbf{E}_c) + \lambda_2 L_{\text{gp}}(\tilde{\mathbf{y}}) \ . \tag{13}$$

To train BE-SNNs, we minimize the average loss functions across all ensemble members, the loss function of BE-SNNs is:

$$L_{\text{BE-SNNs}} = \frac{1}{N_e} \sum_{n_e=1}^{N_e} L_{n_e} \ . \tag{14}$$

OOD detection score. Since softmax activations can produce arbitrarily high confidence even for OOD samples (Liu et al., 2020), we propose to adopt an adaptively tempered variant of softmax activation in the BE-SNNs for detecting OOD samples. Notably, this modified version of softmax is only used for OOD detection, but not for training and making predictions for in-distribution samples.

Given a BE-SNN-1, denote by $\bar{\mathbf{d}}(\mathbf{x}) = [\bar{\mathbf{d}}_1(\mathbf{x}), \cdots \bar{\mathbf{d}}_C(\mathbf{x})]$ the distance vector for each class, we compute a categorical distribution by using the tempered negative distance

$$\xi(\mathbf{x}) = \frac{-\bar{\mathbf{d}}(\mathbf{x})}{\exp(\min_c(\bar{\mathbf{d}}_c(\mathbf{x})))} \tag{15}$$

as the input of a standard softmax function $\sigma(\cdot)$, where $\min_c(\bar{\mathbf{d}}_c(\mathbf{x}))$ is the smallest distance between class features and input features. It is worth mentioning that $\sigma(\xi(\mathbf{x}))$ can be seen as a tempered softmax prediction as in (Guo et al., 2017), whereas the tempering constant $\exp(\min_c(\bar{\mathbf{d}}_c(\mathbf{x})))$ is data-adaptive.

An intuitive interpretation of the above modification is that the further an input feature is from class features, the more uniform the categorical distribution should be. This is realized by the introduction of the adaptive tempering constant $\exp(\min_c(\bar{\mathbf{d}}_c(\mathbf{x})))$ to avoid producing arbitrarily high confidence on OOD samples.

We propose to use the entropy of the tempered softmax prediction $\mathbb{H}[\zeta(\mathbf{x})] = -\sum_c \bar{\omega}_c \log \bar{\omega}_c$ as the OOD score, where

$$\zeta(\mathbf{x}) = [\bar{\omega}_1, \cdots, \bar{\omega}_C] \ , \tag{16}$$

$$\bar{\omega}_c = \frac{1}{N_e} \sum_{n_e=1}^{N_e} \sigma_c(\xi_{n_e}(\mathbf{x})) \ , \tag{17}$$

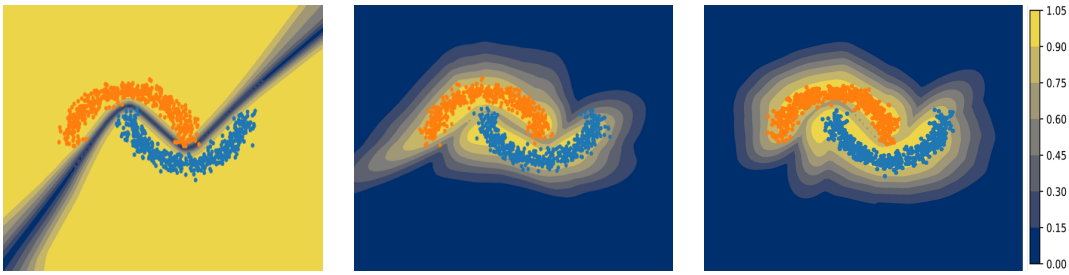
and $\xi_{n_e}(\mathbf{x})$ is the tempered negative distance of the n_e-th ensemble member computed in Equation (15). The decision function in the BE-SNNs is as follows:

$$\mathcal{S}(\mathbf{x}) = \begin{cases} \text{out-of-distribution} & \text{if} \quad \mathbb{H}[\zeta(\mathbf{x})] \geq \tau \\ \text{in-distribution} & \text{if} \quad \mathbb{H}[\zeta(\mathbf{x})] < \tau \end{cases} , \tag{18}$$

where τ is a user-specified threshold.

4 Experiments

In this section, we evaluate the performance of BE-SNNs on several OOD benchmarks adopted by previous works (van Amersfoort et al., 2020; Wan et al., 2018), including the Two-Moons dataset, the FashionMNIST vs MNIST dataset, the FashionMNIST vs NotMNIST dataset, and the CIFAR10 vs SVHN dataset. In all the experiments, we set the regularization coefficient λ_1 in Equation (13) to be 1.0, and set λ_2 to be 0.5. Details on network architectures employed in the experiment can be found in Appendix A.



(a) Deep ensemble (b) DUQ (c) BE-SNN-1

Figure 1: Visualized experimental results of deep ensemble, DUQ, and BE-SNN-1 on the Two-Moons dataset. The confidence of models in their predictions is indicated by different colours, where blue corresponds to low confidence, and yellow shows high confidence. Left plot shows that deep ensemble only has low confidence on regions around its decision boundary. In contrast, BE-SNN-1 only produce high confidence predictions on the regions where the model was trained on, and show gradually decreasing confidence on the other areas far away from the training data. DUQ produces slightly more mismatch between the high confidence regions and the samples which the model was trained with compared to the BE-SNN-1.

4.1 Two Moons dataset

Following the setup in (van Amersfoort et al., 2020), we first visualize the performance of the BE-SNN-1 on the Two-Moons dataset and compare it with the DUQ and a deep ensemble of softmax networks with ensemble size of 4. As shown in Figure 1, the Two-Moons dataset is a 2-dimensional classification dataset. In this experiment, while the evaluated dataset is a toy dataset, we do not only focus on the classification accuracy since both models can achieve 100% accuracy, but also how well the tested models can assign proper confidence scores to their predictions. Notably, for the DUQ and BE-SNN-1, we consider the exponentiated distance to the closet centroid as the confidence of the prediction, and the confidence of deep ensemble methods is evaluated by the maximum value of the averaged softmax prediction.

The experiment results presented in Figure 1 indicate that, even with deep ensembles of softmax network, it can still produce overconfident predictions on OOD samples. A possible reason is that the Two-Moons dataset is too simple for the networks to converge to different local optima, thus all ensemble members are almost the same after training. As a comparison, both the BE-SNN-1 and the DUQ are able to assign proper confidence to their predictions, since it can be observed from Figure 1 that they only produce high confidence predictions on the in-distribution, and show gradually decreasing confidence on the other areas far away from the in-distribution.

In-Distibution	Model	OOD	Accuracy↑	FPR95↓	AUROC↑	AUPRC↑
FashionMNIST	BE-SNNs	MNIST	**92.4%±0.1%**	**0.169±.016**	**0.961±.016**	**0.989±.002**
		NotMNIST		**0.182±.057**	**0.970±.018**	0.988±.005
	DUQ	MNIST	92.1%±0.1%	0.244±0.043	0.944±0.013	0.975±.004
		NotMNIST		0.231±0.047	0.953±0.009	**0.989±.007**
	Gaussian	MNIST	92.0%±0.1%	0.274±0.041	0.932±0.014	0.980±.008
		NotMNIST		0.281±0.061	0.940±0.012	0.982±.003
	Softmax	MNIST	92.3%±0.2%	0.564±0.051	0.889±0.034	0.930±.015
		NotMNIST		0.531±0.121	0.910±0.013	0.941±.009
CIFAR-10	BE-SNNs	SVHN	93.5%± 0.1%	**0.359±.031**	**0.940±0.017**	**0.982±.007**
	DUQ	SVHN	93.7%± 0.2%	0.452±.035	0.921±0.013	0.970±.006
	Gaussian	SVHN	93.1%± 0.1%	0.492±.045	0.919±0.009	0.975±.003
	Softmax	SVHN	**93.9%± 0.1%**	0.680±.032	0.872±0.024	0.924±.011

Table 1: Experiments results on FashionMNIST vs MNIST, FashionMNIST vs NotMNIST, and CIFAR-10 vs SVHN datasets. Compared baselines are the DUQ (van Amersfoort et al., 2020), and the Gaussian classifier (Wan et al., 2018). The lower the FPR95, the higher the AUROC and the AUPRC, the better the performance on the OOD detection task. The mean and standard deviation are calculated over 5 random seeds.